

Referencia técnica

TDS Toolkit

Escribir un decodificador de protocolo

Para osciloscopios digitales de las series TDS 500, TDS 600 y TDS 700

Documento TDSTK-DEC-01

Copyright © 2026 Jared Cabot. Licencia MIT.

TDSTK-DEC-01

Copyright © 2026 Jared Cabot. Todos los derechos reservados.

Este documento describe la interfaz de complementos decodificadores de TDS Toolkit. La información de esta publicación reemplaza a la de todo el material publicado anteriormente.

TDS Toolkit es software anfitrión; no es un producto de Tektronix. Tektronix y TDS se usan solo para nombrar los instrumentos que el software admite.

Este software se publica bajo la Licencia MIT. EL SOFTWARE SE PROPORCIONA "TAL CUAL", SIN GARANTÍA DE NINGÚN TIPO, EXPRESA O IMPLÍCITA. El texto completo acompaña al programa como LICENSE.

Índice

Manuales relacionados	iv
1 El modelo de decodificador	1-1
Qué es un decodificador	1-1
Un archivo, un protocolo	1-1
El contrato	1-1
Descubrimiento	1-1
Dónde residen los archivos	1-1
2 El Signal	2-1
Qué recibe una decodificación	2-1
Muestras y temporización	2-1
Niveles, no muestras	2-1
De dónde viene un Signal	2-2
3 El contrato del decodificador	3-1
Atributos de clase	3-1
Los cuatro atributos	3-1
Roles y fuentes	3-1
Los métodos	3-2
decode	3-2
Parámetros	3-3
El tipo Param	3-3
4 Frames y representación	4-1
El Frame	4-1
Un evento decodificado	4-1
Estado	4-1
Representar el resultado	4-2
Cuatro métodos pequeños	4-2
Color	4-2

5 Construir un decodificador	5-1
Un ejemplo resuelto	5-1
El protocolo	5-1
Los pasos	5-1
El código	5-1
Importaciones y atributos	5-1
Recuperar los bits	5-2
Agrupar las palabras	5-2
Representación	5-3
El atajo serie	5-4
No reimplemente el muestreo de bits	5-4
6 Instalación y prueba	6-1
Instalación	6-1
Colocar y abrir	6-1
Prueba	6-1
Accionarlo con un Signal sintético	6-1
Pruebe todos los decodificadores	6-2
Lista de comprobación	6-3
Antes de confiar en él	6-3

Índice de tablas

Tabla 1–1: Archivos del subsistema de decodificación	1–2
Tabla 2–1: Miembros de un Signal	2–2
Tabla 3–1: Atributos de clase de Decoder	3–1
Tabla 3–2: Métodos de Decoder	3–2
Tabla 3–3: Tipos de Param	3–3
Tabla 4–1: Campos de Frame	4–1
Tabla 4–2: Bits de estado de Frame	4–2

Prefacio

Este manual describe cómo añadir un decodificador de protocolo a TDS Toolkit. Un decodificador es un único archivo de Python. Lee una forma de onda capturada y devuelve una lista de eventos decodificados; el programa lo enumera, asigna sus entradas, lo ejecuta y dibuja el resultado sin ningún código adicional.

Se presupone que el lector es un programador con experiencia en Python. No hace falta conocer el resto del programa: toda la interfaz es una clase base y tres pequeños tipos de datos, todos en el módulo `tds_decode.py`.

Lea este manual junto a dos archivos que se incluyen con el programa: `decoders/_template.py`, el esqueleto que copiar, y `decoders/README.txt`, la versión abreviada de lo que sigue.

Manuales relacionados

La interfaz, y los archivos que leer junto a este manual:

- `decoders/_template.py` - el esqueleto que copiar.
- `decoders/README.txt` - la versión abreviada de lo que sigue.
- `tds_decode.py` - el motor: la clase base `Decoder` y los tipos `Signal`, `Frame` y `Param`.

El modelo de decodificador

Qué es un decodificador

Un archivo, un protocolo

Un decodificador es un único archivo de Python en la carpeta decoders que define una subclase de Decoder. Cada archivo es un protocolo. La pestaña Formas de onda enumera todos los decodificadores que encuentra, así que añadir un protocolo supone un solo archivo y no cambia nada más en el programa. Una decodificación se ejecuta por completo sobre una forma de onda capturada que la pestaña Formas de onda ya tiene: un canal leído del instrumento, o una referencia almacenada cargada de un archivo. Nada en un decodificador toca el instrumento. Ese es el propósito del diseño. Funciona en cualquier TDS, incluidos los modelos base cuyo firmware no puede ejecutar una aplicación propia de decodificación de buses, porque la decodificación ocurre en el host, sobre las muestras, después de la captura.

El contrato

Toda la interfaz es la clase base Decoder. Un decodificador establece cuatro atributos de clase e implementa un método. Los otros tres métodos tienen valores predeterminados funcionales, que solo se sobrescriben para representar los campos propios del protocolo en la tabla y en la gráfica. Los tres tipos de datos que maneja un decodificador —Signal, Frame y Param— se describen en los capítulos que siguen.

Al iniciarse, y cada vez que se abre la pestaña Formas de onda, el programa examina la carpeta decoders en busca de archivos *.py, importa cada uno de forma aislada e instancia todas las subclases de Decoder que encuentra. Un archivo cuyo nombre empieza por un guion bajo se trata como un auxiliar y se omite, y así es como _template.py sigue siendo una plantilla y nunca aparece como un protocolo vacío en la lista.

Descubrimiento

La importación es defensiva. Un archivo que no se importa, ya sea por un error de sintaxis o por una referencia incorrecta, se omite —con una nota escrita en el registro del programa— en lugar de arrastrar consigo toda la lista. Un decodificador a medio terminar nunca oculta los que funcionan. Cuando dos carpetas ofrecen un decodificador con el mismo NAME, gana la carpeta posterior, así que una copia escrita localmente reemplaza a la suministrada.

Dónde residen los archivos

Los decodificadores que se incluyen con el programa residen en una carpeta `decoders` dentro de la instalación. Se mantiene una copia modificable junto al archivo de ajustes, `tdstoolkit.json`, que se completa a partir del conjunto suministrado en cada ejecución, exactamente igual que la biblioteca de máscaras. Un decodificador añadido en una compilación posterior se copia sin sobrescribir uno escrito localmente; un decodificador suministrado que se haya eliminado vuelve en la siguiente ejecución, porque el conjunto suministrado debe estar presente. Coloque un archivo nuevo en la carpeta modificable y aparecerá en la lista la próxima vez que se abra la pestaña. No hace falta recompilar.

Tabla 1-1: Archivos del subsistema de decodificación

Archivo	Función
<code>tds_decode.py</code>	El motor: la clase base <code>Decoder</code> , los tipos <code>Signal</code> , <code>Frame</code> y <code>Param</code> , las constantes de estado, el auxiliar serie compartido y el descubrimiento. Un decodificador importa de él y no lo modifica.
<code>decoders/_template.py</code>	El esqueleto. Cópielo para empezar un decodificador nuevo.
<code>decoders/README.txt</code>	Las instrucciones abreviadas y un mapa de los decodificadores de ejemplo.
<code>decoders/.py</code>	Un protocolo cada uno: <code>uart.py</code> , <code>i2c.py</code> , <code>spi.py</code> , <code>can.py</code> y los demás. Lea el más cercano al protocolo que se va a añadir.

El Signal

Qué recibe una decodificación

	Al método <code>decode()</code> se le entrega un <code>Signal</code> por cada entrada que el protocolo necesita. Un <code>Signal</code> es un registro capturado reducido a lo que un decodificador necesita: las muestras enteras y la temporización que convierte un índice de muestra en un tiempo. No lleva voltios ni escalado de visualización. Un decodificador compara una muestra con un umbral y nunca necesita conocer la tensión, así que las cuentas brutas del digitalizador se transmiten sin escalar y un registro de un cuarto de millón de puntos se entrega sin coste alguno.
Muestras y temporización	El umbral de decisión, la banda muerta de histéresis y la amplitud de la señal se calculan antes de llamar a <code>decode()</code>. Un decodificador no calcula un umbral propio. El umbral es el punto medio de los extremos del registro; la histéresis es un octavo de la amplitud, lo bastante ancha para rechazar las oscilaciones del flanco sin que un flanco lento deje de cruzarla. Estos valores se leen, no se establecen.
Niveles, no muestras	Para la mayoría de los protocolos, la forma útil del registro no son las muestras brutas sino un nivel lógico limpio por muestra. El método <code>digitize()</code> devuelve exactamente eso: un 0 o un 1 por cada muestra, con la propia histéresis del registro aplicada para que las oscilaciones del flanco no se lean como varios cruces. Llámelo una vez y recorra el resultado. Un protocolo que necesita las cuentas brutas, por ejemplo una línea bipolar con dos umbrales, lee <code>sig.data</code> directamente en su lugar.

Tabla 2-1: Miembros de un Signal

Miembro	Significado
sig.data	Las muestras enteras, una secuencia indexable de cuentas brutas del digitalizador.
sig.n	El número de muestras, igual que len(sig.data).
sig.dt	Segundos por muestra.
sig.t0	El tiempo de la primera muestra, en segundos relativos al disparo.
sig.first_index	El desplazamiento en el registro de la primera muestra. Súmelo a un índice de muestra local para obtener una posición absoluta en el registro.
sig.threshold	El nivel de decisión, en cuentas. Lo establece el framework.
sig.hysteresis	La banda muerta en torno al umbral, en cuentas. La establece el framework.
sig.span	La diferencia entre la muestra más alta y la más baja del registro. La establece el framework; una amplitud cercana a cero indica una fuente plana y sin señal.
sig.time_at(i)	El tiempo en segundos en el índice de muestra local i, es decir $t_0 + i \times dt$.
sig.digitize()	Un nuevo bytearray con un nivel 0/1 por muestra, con histéresis.
sig.auto_threshold()	Recalcula threshold, hysteresis y span. El framework ya lo llama; invóquelo solo en un Signal construido a mano (véase la Sección 6).

NOTA. Un Signal lleva cuentas brutas, no voltios. Compare con sig.threshold, nunca con una tensión. Un decodificador que lee correctamente a una escala vertical lee entonces correctamente a cualquier otra escala, porque el umbral se mueve con el registro.

De dónde viene un Signal

El Signal que recibe decode() lo construye el programa, no el decodificador, a partir de la forma de onda capturada con el método de clase Signal.from_waveform(). Lee los niveles enteros y el intervalo de muestreo y el desfase de disparo del preámbulo de la forma de onda, construye el Signal y llama a auto_threshold() sobre él antes de entregárselo al decodificador. Un Signal se construye a mano solo al escribir una prueba, que trata la Sección 6.

El contrato del decodificador

Atributos de clase

Una subclase de decodificador establece cuatro atributos de clase. NAME y DESCRIPTION la nombran en la lista de protocolos. SOURCES declara las entradas de forma de onda que necesita. PARAMS declara los ajustes que ofrece la pestaña. Todo lo demás se hereda.

Tabla 3–1: Atributos de clase de Decoder

Atributo	Significado
NAME	El nombre del protocolo. Se muestra en la lista y se usa como título de la etiqueta de la gráfica. Déjelo en blanco, o "?", y el cargador no lista la clase; así es como la plantilla queda fuera de la lista.
DESCRIPTION	Una línea para la segunda columna de la lista.
SOURCES	Una lista de nombres de rol, en orden. Un solo rol es un protocolo de un hilo; con dos o más, a cada uno se le asigna una fuente capturada. Un "?" final marca un rol opcional que puede quedar sin asignar.
PARAMS	Una lista de objetos Param, un control cada uno. Puede estar vacía.
COLOUR	El color del galón para una trama limpia, como "#rrggb". Un decodificador define sus propios colores; el programa no tiene ningún ajuste de color para los decodificadores. Establézcalo para dar un solo color a todo el protocolo.
FAULT_COLOUR	El color del galón para una trama defectuosa. Rojo de forma predeterminada, para que un error destaque sobre el color de cualquier protocolo.

Roles y fuentes

SOURCES es tanto el número de formas de onda que un protocolo necesita como sus nombres. Un protocolo de un hilo declara un rol, por convención "Line". Un protocolo de dos hilos como I2C declara ["SDA", "SCL"]; la pestaña muestra un selector de fuente por rol, y decode() recibe un Signal por cada uno. Un signo de interrogación final, como en el ["CLK", "MOSI", "MISO?", "CS?"] de SPI, marca un rol que el operador puede dejar sin asignar; su Signal está entonces ausente del diccionario que recibe decode(), y el código de decode() lo comprueba.

Los métodos

decode `decode(self, signals, params)` es el único método obligatorio. `signals` es un diccionario que asigna cada nombre de rol, sin el "?" final, a su `Signal`; un rol opcional que no se asignó está ausente. `params` es un diccionario que asigna cada clave de `Param` al valor que eligió el operador. El método devuelve una tupla de dos elementos: una lista de objetos `Frame` y una cadena de nota de una línea.

La nota se muestra encima de la tabla de resultados. Úsela para informar de lo que dedujo la decodificación, como la velocidad de bits detectada, las muestras por bit y el número de tramas encontradas, o, cuando el registro no puede leerse, para explicar por qué. En caso de fallo, devuelva una lista de tramas vacía y una nota que explique el problema en una frase.

Tabla 3–2: Métodos de Decoder

Método	Devuelve
<code>decode(signals, params)</code>	(frames, note). La decodificación en sí. Obligatorio.
<code>columns()</code>	Los encabezados de la tabla después de la columna Tiempo, como una lista de pares (encabezado, ancho_en_píxeles). Sobrescríbalos para nombrar los campos del protocolo.
<code>row(frame, ascii)</code>	Las celdas de la tabla para una trama, que corresponden a <code>columns()</code> después del tiempo. Sobrescríbalos para dar formato a los campos del protocolo.
<code>label(frame, ascii)</code>	La cadena útil más corta para el galón en la gráfica. Sobrescríbalos para mantenerlo escueto.
<code>colour(frame)</code>	El color del galón para una trama. De forma predeterminada, <code>COLOUR</code> para una trama limpia y <code>FAULT_COLOUR</code> para una defectuosa. Sobrescríbalos para colorear por galón, según el valor de <code>frame.kind</code> .

El argumento `ascii` que se pasa a `row()` y `label()` es el estado del conmutador Hex / ASCII de la pestaña. Respételo donde tenga sentido —un byte mostrado como carácter o como dos dígitos hexadecimales— e ignórelo donde no lo tenga.

Parámetros

Cada Param de la lista PARAMS se convierte en un control de la pestaña, de modo que un protocolo añada sus propias opciones sin código de interfaz de usuario. El constructor es Param(key, label, kind, choices, default, lo, hi, unit, note). key es la clave del diccionario que lee decode(); label es el rótulo que ve el operador; kind es una de tres cadenas.

Tabla 3–3: Tipos de Param

kind	Control y campos
"choice"	Una lista desplegable. choices es una lista de pares (value, label); el value es lo que llega a params, el label es lo que se muestra. default es uno de los valores.
"int"	Un campo de entrada de enteros. lo y hi lo acotan; unit es un sufijo del rótulo como "bits". Por convención, un default de 0 se interpreta como "auto", lo que debería indicar la nota.
"bool"	Una casilla de verificación. default es True o False.

El tipo Param

Cuando se omite default, toma el valor de la primera opción para un choice, False para un bool y 0 para un int. La nota opcional se muestra junto al control como indicación.

Frames y representación

El Frame

Un evento decodificado

Un Frame es un evento decodificado: un carácter UART, un byte I2C, una palabra SPI, una condición del bus. El tipo es plano y compartido por todos los protocolos. Lleva un dato primario, uno secundario opcional, una longitud, un estado y un tipo (kind). Lo que significan los campos es asunto del decodificador, ya que solo el decodificador que creó una trama la representa, pero su forma es fija.

Construya un Frame con argumentos de palabra clave; cada campo tiene un valor predeterminado. Los dos campos de índice son los más importantes y aquellos en los que es más fácil equivocarse. `index` y `end_index` son posiciones absolutas de muestra en el registro: la gráfica dibuja el recuadro del galón de una a la otra. Fórmelos como `sig.first_index` más el índice de muestra local, igual que el tiempo se obtiene de `sig.time_at()` del índice local.

Tabla 4-1: Campos de Frame

Campo	Significado
time	El tiempo de inicio del evento en segundos, de <code>sig.time_at(local_index)</code> .
index	La muestra de inicio absoluta: <code>sig.first_index</code> + inicio local.
end_index	La muestra final absoluta: <code>sig.first_index</code> + fin local. El recuadro abarca de <code>index</code> a <code>end_index</code> .
value	El dato primario: el byte, la palabra, el valor medido.
tag	Un dato secundario opcional: una dirección I2C, una palabra MISO de SPI, una frecuencia medida. 0 si no se usa.
length	La longitud del dato, en la unidad que cuente el protocolo. Su valor predeterminado es 1.
status	Un OR bit a bit de las constantes de estado; OK (cero) cuando la trama está limpia.
kind	Una cadena libre, como "byte", "addr", "data", "start" o "stop", que <code>row()</code> y <code>label()</code> usan para distinguir y representar cada parte de una trama por sí misma.

Estado status es una máscara de bits. Una trama limpia es OK, que es cero. Una trama con problemas combina mediante OR una o más de las constantes de la Tabla 4-2. El framework dibuja cualquier trama cuyo estado sea distinto de cero como un error —un galón rojo y un signo de exclamación inicial en la etiqueta—, sin que el decodificador haga nada más que establecer el bit.

Tabla 4-2: Bits de estado de Frame

Constante	Valor	Significado
OK	0	Limpia; sin error.
FRAMING_ERROR	1	La trama no se cerró como exige su formato, como un bit de parada UART que no fue una marca.
PARITY_ERROR	2	Un bit de paridad no concordó con los datos.
TRUNCATED	4	El registro terminó a mitad de la trama.
ACK_ERROR	8	Un byte que esperaba un reconocimiento no fue reconocido.

Importe las constantes que use desde `tds_decode` junto con `Decoder`, `Frame` y `Param`. No defina valores de estado más allá de estos; los cinco anteriores son los que entiende el renderizador.

Representar el resultado

Cuatro métodos pequeños

La clase base representa una trama como un byte simple: hexadecimal, ASCII y una palabra de estado. Sobrescriba `columns()`, `row()` y `label()` para usar en su lugar los términos propios del protocolo. `columns()` devuelve los encabezados después de la columna Tiempo, cada uno con un ancho en píxeles. `row()` devuelve una celda por encabezado, en el mismo orden. `label()` devuelve la cadena útil más corta para el galón dibujado sobre la propia forma de onda.

Un protocolo cuyas tramas llevan tipos distintos distingue según `frame.kind` en estos métodos, de modo que un START, un byte de dirección y un byte de datos se lean cada uno como lo que son en la tabla y en la gráfica. Mantenga `label()` escueto: se dibuja dentro de un pequeño recuadro sobre la traza, así que de uno a tres caracteres es la longitud adecuada.

Color

El cuarto método, `colour(frame)`, fija el color del galón, y el decodificador lo controla por completo: el programa no tiene ningún ajuste de color para los decodificadores. De forma predeterminada, pinta una trama limpia con el `COLOUR` de la clase y una defectuosa con `FAULT_COLOUR`, así que fijar `COLOUR` da un solo color a todo el protocolo. Sobrescriba `colour()` y distinga según `frame.kind` para pintar un color por galón, de modo que la dirección, los datos y el tramado de una línea se lean cada uno en un color propio. Devuelva un `"#rrggbb"`; recurra a la clase base para los tipos que el decodificador no colorea por sí mismo.

Construir un decodificador

Un ejemplo resuelto

El protocolo

Este capítulo construye un decodificador completo a partir de la plantilla. El protocolo de ejemplo es un único hilo autosincronizado que transporta un bit por pulso: la línea sube, permanece alta durante una fracción del periodo de bit y baja; un alto corto es un 0 y un alto largo es un 1, decidido según si el tiempo en alto supera la mitad del periodo. Los bits se agrupan en palabras de un tamaño y un orden de bits elegidos. Es la familia de la línea de píxeles WS2812 y del mando infrarrojo NEC, y no está entre los protocolos integrados, así que es un decodificador genuinamente nuevo y no una copia de otro.

Los pasos

1. *Copie la plantilla:* Copie `decoders/_template.py` a `decoders/pulsebits.py`.
2. *Nombre el protocolo:* Cambie el nombre de la clase y establezca `NAME`, `DESCRIPTION` y `SOURCES`.
3. *Declare los ajustes:* Rellene `PARAMS` con los controles de tamaño de palabra y orden de bits.
4. *Escriba la decodificación:* Recupere los bits de la pista de niveles y luego agrúpelos en palabras.
5. *Represente los campos:* Sobrescriba `columns()`, `row()` y `label()` para mostrar la palabra en hexadecimal y binario.
6. *Pruébalo:* Ejecute el decodificador con un Signal sintético y verifique el resultado, como muestra la Sección 6.

El código

Importaciones y atributos

La cabecera importa solo desde `tds_decode`, nombra la clase y declara las dos entradas que ofrecerá la pestaña: un tamaño de palabra y un orden de bits. `SOURCES` contiene un rol, así que la pestaña muestra un único selector de fuente.

```

from tds_decode import Decoder, Frame, Param
from tds_decode import OK, TRUNCATED

class PulseBitsDecoder(Decoder):

    NAME = "Pulse-width bits"
    DESCRIPTION = "Self-clocked pulse-width line, one wire"

    SOURCES = ["Line"]
    PARAMS = [
        Param("bits", "Word size", "int", default=8,
              lo=1, hi=32, unit="bits"),
        Param("order", "Bit order", "choice",
              choices=[("msb", "MSB first"),
                       ("lsb", "LSB first")],
              default="msb"),
    ]

```

Recuperar los bits

decode() toma el único Signal, se protege frente a una fuente plana y reduce el registro a un nivel 0/1 por muestra con digitize(). Luego recorre la pista de niveles. Cada bit comienza en un flanco de subida; la racha de muestras altas es el pulso, la racha de muestras bajas que sigue continúa hasta el siguiente flanco de subida, y ambas juntas son el periodo de bit. El bit se decide comparando el tiempo en alto con el periodo.

```

def decode(self, signals, params):
    sig = signals["Line"]
    if sig.span < 8:
        return [], "no signal on this source"
    level = sig.digitize()
    n = len(level)
    word_bits = max(1, int(params.get("bits") or 8))
    msb = params.get("order", "msb") == "msb"

    bits = [] # (value, start_index, end_index)
    i = 1
    while i < n:
        if level[i] and not level[i - 1]: # a rising edge
            start = i
            while i < n and level[i]: # the high pulse
                i += 1
            high = i - start
            while i < n and not level[i]: # low, to next rise
                i += 1
            period = i - start
            if period > 0:
                bit = 1 if high * 2 > period else 0
                bits.append((bit, start, i))
        else:
            i += 1

```

Agrupar las palabras

Los bits se toman entonces en grupos de `word_bits`, con el más o el menos significativo primero según indique el parámetro. Un grupo que queda incompleto porque el registro terminó se emite igualmente, con su estado marcado como `TRUNCATED` para que el renderizador lo dibuje como un error. Cada Frame toma su tiempo del primer bit y abarca desde el inicio del primer bit hasta el fin del último. La nota informa de los totales.

```
frames = []
for base in range(0, len(bits), word_bits):
    chunk = bits[base:base + word_bits]
    value = 0
    for pos, (bit, _s, _e) in enumerate(chunk):
        shift = (word_bits - 1 - pos) if msb else pos
        value |= bit << shift
    start_local = chunk[0][1]
    end_local = chunk[-1][2]
    ok = len(chunk) == word_bits
    frames.append(Frame(
        time=sig.time_at(start_local),
        index=sig.first_index + start_local,
        end_index=sig.first_index + end_local,
        value=value, length=len(chunk),
        status=OK if ok else TRUNCATED,
        kind="data"))
return frames, "%d bit(s), %d word(s)" % (
    len(bits), len(frames))
```

Representación

Los tres métodos de representación muestran la palabra en hexadecimal y en binario de su propia anchura de bits, y marcan una palabra truncada. `label()` se limita a dos dígitos hexadecimales con una exclamación inicial en caso de error, que es la convención que siguen los decodificadores integrados.

```
def columns(self):
    return [("Time", 92), ("Hex", 60),
            ("Binary", 80), ("Status", 70)]

def row(self, frame, ascii=True):
    return ["0x%02X" % frame.value,
            format(frame.value, "%0db" % frame.length),
            "OK" if frame.status == OK else "CUT"]

def label(self, frame, ascii=True):
    text = "%02X" % (frame.value & 0xFF)
    return ("!" + text) if frame.status else text
```

Ese archivo es un decodificador completo, que el programa ya puede listar. Colocado en la carpeta `decoders`, aparece en la pestaña como "Pulse-width bits" con un cuadro de tamaño de palabra y una lista desplegable de orden de bits, y decodifica cualquier fuente asignada a su única entrada.

El atajo serie

No reimplemente el muestreo de bits

El ejemplo resuelto muestrea los bits por sí mismo porque su protocolo es autosincronizado y distinto de todo lo integrado. Para cualquier cosa de la familia serie asíncrona, como RS-232, RS-422/485, MODBUS, MIDI, LIN, XBee y DMX512, no reimplemente el muestreo de bits. Llame al auxiliar compartido `serial_bytes()`, probado con un corpus de capturas, que realiza la autodetección de baudios, el muestreo en el centro del bit y las comprobaciones de paridad y de bit de parada de la única manera probada, y construya el tramado sobre el flujo de bytes que devuelve.

```
from tds_decode import Decoder, Param
from tds_decode import serial_bytes, STANDARD_BAUDS

class MyUartDecoder(Decoder):

    NAME = "My UART"
    DESCRIPTION = "Async serial, one wire"
    SOURCES = ["Line"]
    PARAMS = [
        Param("baud", "Bit rate", "choice",
              choices=[(0, "Auto")]
              + [(b, str(b)) for b in STANDARD_BAUDS],
              default=0),
        Param("framing", "Framing", "choice",
              choices=[(f, f) for f in
                       ("8N1", "8E1", "8O1", "8N2")],
              default="8N1"),
    ]

    def decode(self, signals, params):
        sig = signals["Line"]
        frames, err, baud, spb = serial_bytes(
            sig, int(params.get("baud") or 0),
            params.get("framing", "8N1"), inverted=False)
        if err:
            return [], err
        return frames, "%d baud, %.1f smp/bit, %d frames" % (
            baud, spb, len(frames))
```

`serial_bytes(sig, baud, framing, inverted)` devuelve cuatro valores: las tramas de byte, una cadena de error que está vacía en caso de éxito, la velocidad en baudios detectada y las muestras por bit. Un baud de 0 le pide que detecte la velocidad. `inverted` en `True` lee un sentido eléctrico de reposo bajo y activo alto, como el RS-232 real. Un protocolo de nivel superior como MODBUS o LIN llama a `serial_bytes` para obtener los bytes en bruto y luego los agrupa y comprueba en sus propias tramas; lea `modbus.py` y `lin.py` para ese patrón.

Instalación y prueba

Instalación

Colocar y abrir

Copie el archivo terminado en la carpeta decoders modificable junto a tdstoolkit.json y abra, o vuelva a abrir, la pestaña Formas de onda. El decodificador aparece en la lista de protocolos. No hay paso de registro ni recompilación. Asigne una fuente capturada a cada uno de sus roles, configure sus parámetros y ejecútelo.

NOTA. Si el decodificador no aparece, es que no se importó. Consulte el registro del programa en busca de una línea que nombre el archivo y la excepción. El cargador omite un archivo que no se importa en lugar de hacer fallar toda la lista, así que un decodificador roto falla de forma silenciosa.

Prueba

Accionarlo con un Signal sintético

Un decodificador es una función pura de sus Signals y sus parámetros, así que se prueba sin instrumento construyendo un Signal a partir de un array de muestras sintético, llamando a decode() y verificando el resultado. Construya las muestras de modo que su significado sea conocido, envuélvalas en un Signal, llame a auto_threshold() —el único paso que normalmente da el framework— y compruebe las tramas.

```

from tds_decode import Signal
from pulsebits import PulseBitsDecoder

def _bit(value, period=40):
    """One pulse-width bit: high for part of the period."""
    high = int(period * (0.66 if value else 0.33))
    return [30000] * high + [-30000] * (period - high)

def demo():
    stream = [-30000] * 20          # idle low lead-in
    for bit in (1, 0, 1, 1, 0, 0, 1, 0): # 0xB2, MSB first
        stream += _bit(bit)
    stream += [-30000] * 20        # idle low tail
    sig = Signal(stream, 1e-6)     # 1 us per sample
    sig.auto_threshold()           # framework does this
    frames, note = PulseBitsDecoder().decode(
        {"Line": sig}, {"bits": 8, "order": "msb"})
    assert len(frames) == 1, note
    assert frames[0].value == 0xB2, "%02X" % frames[0].value
    print("ok:", note)

if __name__ == "__main__":
    demo()

```

Pruebe todos los decodificadores

Todos los decodificadores integrados se comprueban de esta manera. El decodificador RS-232 se ejecuta con un corpus de capturas reales; los demás decodificadores se comprueban con un vector sintético en `tdstest.py`, cada uno con su correspondiente bite de mutación en `bitecheck.py` que rompe el decodificador a propósito y confirma que la prueba lo detecta. Añada un caso de prueba por cada protocolo que añada.



PRECAUCIÓN. Una decodificación que no se prueba es una decodificación que se desviará. Añada un vector sintético por cada decodificador que se agregue, y un bite de mutación que demuestre que el vector es esencial, antes de confiar en el resultado.

Lista de comprobación

Antes de confiar en él

- El archivo tiene un nombre en minúsculas, sin espacios, y no empieza por un guion bajo.
- La clase establece NAME, DESCRIPTION, SOURCES y PARAMS, y NAME no está en blanco.
- decode() devuelve una tupla (frames, note) en todas las rutas, incluidas las de fallo.
- Los índices index y end_index del Frame son absolutos: sig.first_index más el índice de muestra local.
- Un registro ilegible devuelve una lista vacía y una nota que explica el motivo, nunca una excepción.
- columns(), row() y label() coinciden en el número y el orden de los campos.
- Una prueba sintética ejecuta el decodificador y verifica los valores decodificados.